9.Write the python to implement Travelling Salesman Problem

Aim

To develop a Python program to solve the Travelling Salesman Problem (TSP) by finding the minimum cost path that visits each city exactly once and returns to the starting city.

Algorithm

1. Start the program.
2. Read the number of cities from the user.
3. Read the cost matrix representing the distances between the cities.
4. Generate all possible routes starting from the first city.
5. Calculate the total travel cost for each route.
6. Compare the costs of all possible routes.
7. Select the route with the minimum total cost.
8. Display the optimal route and its minimum travel cost.
9. Stop.

Program

```
from itertools import permutations


# Read number of cities
n = int(input("Enter the number of cities: "))


print("Enter the Cost Matrix:")


cost = []
for i in range(n):
    row = list(map(int, input().split()))
    cost.append(row)


cities = list(range(n))
start = 0
```

```
min_cost = float('inf')
best_path = None

# Generate all possible tours
for path in permutations(cities[1:]):
    current_path = (start,) + path
    current_cost = 0

    # Calculate path cost
    for i in range(len(current_path) - 1):
        current_cost += cost[current_path[i]][current_path[i + 1]]

    # Return to starting city
    current_cost += cost[current_path[-1]][start]

    if current_cost < min_cost:
        min_cost = current_cost
        best_path = current_path

# Display result
print("\nOptimal Path:")
for city in best_path:
    print(city, end=" -> ")
print(start)

print("Minimum Cost =", min_cost)
```

Input

```
Enter the number of cities: 4

Enter the Cost Matrix:
0 10 15 20
10 0 35 25
```

```
15 35 0 30
20 25 30 0
```

Output

```
Optimal Path:
0 -> 1 -> 3 -> 2 -> 0
Minimum Cost = 80
>>>
```

Note: If there are multiple optimal routes with the same minimum cost, the program may display any one of them.

Result

The Python program was successfully implemented to solve the Travelling Salesman Problem (TSP). The program accepts the number of cities and the cost matrix from the user, evaluates all possible tours using permutation search, identifies the route with the minimum travel cost, and displays the optimal path along with its minimum cost.